

53rd SME North American Manufacturing Research Conference (NAMRC 53, 2025)

## Retrofitting Legacy Systems for Industry 4.0 via OPC UA and Distributed Control

Nico Holmes, Luka Katavich, Xun Xu\*

*The University of Auckland, Auckland, 1010, New Zealand*\* Corresponding author. *E-mail address:* [x.xu@auckland.ac.nz](mailto:x.xu@auckland.ac.nz)

---

### Abstract

As manufacturing shifts towards Industry 4.0, enterprises are being pushed to invest in smart, Industry 4.0-capable hardware to streamline their production processes. In particular, Small and Medium Enterprises (SMEs) may be hesitant to adopt these new technologies due to the significant capital investment required. To support this, legacy hardware must be upgraded to an Industry 4.0 standard at a minimal cost. This introduces the concept of digital retrofitting. Many methods of digital retrofitting have been proposed in the past. However, many focused on a single aspect and did not focus on a wide range of machines in a production environment. This paper proposes a general digital retrofitting framework that can be applied to a large range of machines, using OPC Unified Architecture (OPC UA) as the standard for communication. The framework is applied in a layered approach, consisting of the asset layer, integration layer, and communication layer, following RAMI4.0 naming conventions. The framework is laid out in detail, focusing on giving individual machines control such that modular, distributed control is achieved. This work then implements the framework on a system consisting of two conveyors and two KUKA robots in the Laboratory for Industry 4.0 Smart Manufacturing Systems (LISMS) and discusses the results and solutions to any problems faced.

© 2025  The Authors. Published by ELSEVIER Ltd. This is an open access article under the CC BY-NC-ND license (<https://creativecommons.org/licenses/by-nc-nd/4.0>)

Peer-review under responsibility of the scientific committee of the NAMRI/SME.

**Keywords:** Industry 4.0, smart manufacturing, OPC UA, distributed control, legacy systems

---

### 1. Introduction

Since it was first coined in 2011, the term ‘Industry 4.0’ has gained significant traction in the manufacturing sector. Otherwise known as the fourth industrial revolution, Industry 4.0 refers to industrial automation that utilizes state-of-the-art intelligent networks, communication, and information to achieve a ‘smart factory.’ These advanced capabilities enable more dynamic and reconfigurable manufacturing that can efficiently produce a wide range of products in small batch sizes, more competitively with mass-produced goods [1]. This manufacturing approach supports Small and Medium Enterprises (SMEs) that do not

meet the required production volumes for traditional automation solutions to be suitable.

While Industry 4.0 is theoretically a transformative technology, the challenge lies in finding methods to implement these technologies economically. Ideally, new systems built with Industry 4.0 capabilities would be installed. However, SMEs are hesitant to adopt these new technologies due to the significant capital investment required [2]. Additionally, businesses may have already invested heavily in their existing automated systems. For this reason, upgrading machines through digital retrofitting is a practical and economical solution, allowing businesses to modernize without sacrificing their current investments [3].

Digital retrofitting is the process of introducing hardware, software, and network capabilities such that a legacy machine can produce additional operational data and be part of the wider IoT infrastructure in the smart factory [4], [5]. These improved capabilities align hardware with Industry 4.0 goals by establishing Machine-to-Machine (M2M) communications and adaptive operation that responds to the environment around it and varying task requirements.

This paper proposes a structured process for retrofitting legacy machines and incorporating them into a larger communications network.

The rest of this paper is organized as follows: Section 2 provides a detailed review of related literature. Section 3 outlines the methodology behind developing and implementing the digital retrofit process on legacy machines. Section 4 outlines how the digital retrofitting process was applied to the machines in the LISMS and explains how each device is configured to meet the communication architecture. Section 5 discusses the results of the digital retrofitting process and communication architecture. Conclusions are presented in Section 6.

## 2. Related Work

This section analyzes the key concepts and technologies behind the digital retrofitting of legacy machines to improve Industry 4.0 capabilities and reviews related work in this field.

### 2.1. Open Process Control Unified Architecture (OPC UA)

To establish Industry 4.0 capabilities, adopting a communication protocol that ensures high interoperability and security is essential. OPC UA has emerged as a widely accepted standard for Industry 4.0, enabling seamless, manufacturer-independent data exchange crucial to the Industrial Internet of Things [6]. The VDMA (the largest industrial association in Europe) lists OPC UA as “a key prerequisite for the successful introduction of Industry 4.0 into production” [7], [8].

Within OPC UA, there are multiple ways of structuring communications. These structures outline how data is transmitted between machines, along with which entities (clients or servers) communicate with each other [9].

Fig. 1 shows the standard OPC UA communication patterns. The simplest approach is the client-server structure, which forms the foundation of all OPC UA communications. This involves a client passing a request to the server, which responds with the appropriate answer, whether that is a data packet, the callback of a method, or an error message [9]. As the simplest form of OPC UA communications, it is easy to implement but offers low scalability and provides just one-way communication.

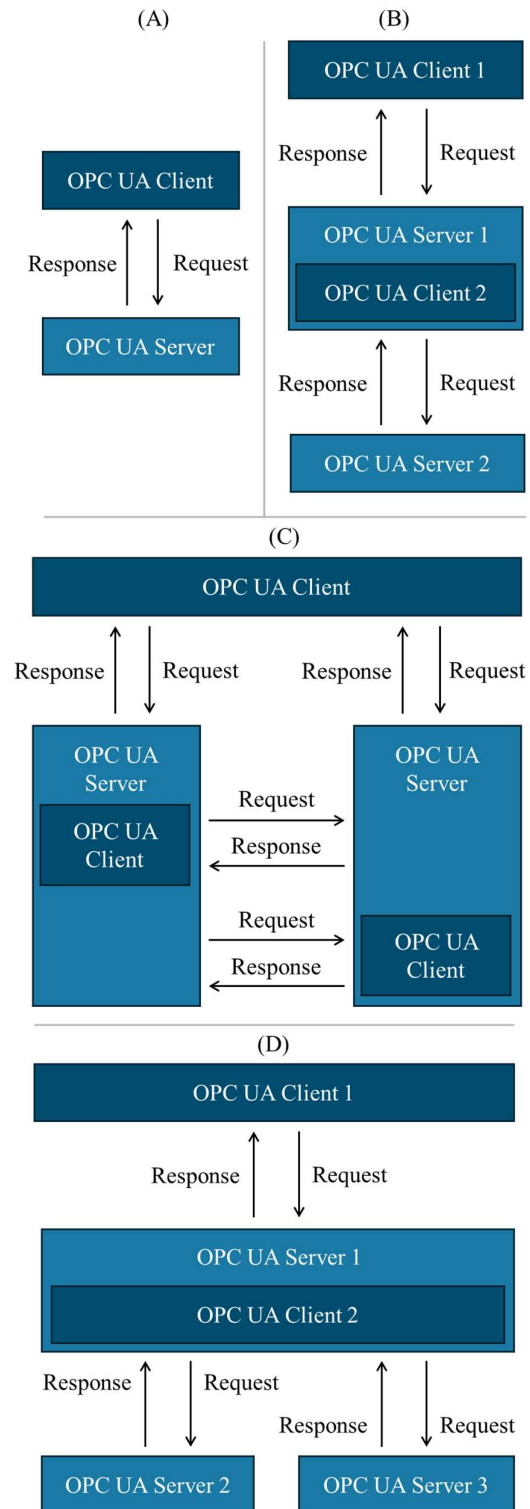


Fig. 1. Communication architecture structures for OPC UA: (a) client-server structure, (b) chained server structure, (c) server-to-server structure, and (d) server aggregation structure [9], [10]

The chained server structure has a central OPC UA server with embedded clients. This allows communication to both other servers and clients, improving scalability, but

communication must still go through the central OPC UA server, which has pathways to all devices [9].

Server-to-server communication requires an OPC UA server for each device, including embedded clients. This allows direct two-way communication between any devices on the network, as all machines can initiate and respond to requests [9].

The server aggregation structure compiles the data of multiple servers into one. This heavily simplifies the network architecture and improves scalability since all data access is through one server rather than a complex web [9]. The main limitation of this architecture is its reliance on the aggregate server for all communication, which introduces a single point of dependency. Additionally, communication may be slower than alternative methods, as direct interfacing is not supported.

## 2.2. Digital Retrofitting Process

To successfully impart improved Industry 4.0 intelligence, digitalization, and communication onto legacy devices, additional sensors and computational hardware may be required. This process can be defined as digital retrofitting. Digital retrofitting focuses on transforming legacy devices into cyber-physical systems utilizing these technologies [4]. This method can be differentiated from traditional retrofitting, which can be categorized as modifying the mechanical properties of hardware to improve its production capabilities [5].

The process of carrying out a digital retrofit involves the development of a ‘technology-agnostic solution that meets the requirements of Industry 4.0’ [10]. This process focuses on the development of an Asset Administration Shell (AAS) [10], which is described as a structured, virtual representation of the software, hardware, and data related to a physical asset. To create this AAS, the steps defined involve selecting a communication architecture, designing an IT platform, designing an information model, and finally defining views and references on information in the wider system.

To handle the increased computational demands of data collection, processing, and transfer associated with Industry 4.0, a control board is often retrofitted [11], [12], [13]. This is a minimally invasive method, essentially applying an Industry 4.0 wrapper to the hardware. This retrofit can be carried out affordably by utilizing a control board such as a Raspberry Pi. A Raspberry Pi offers a wide range of connectivity options such as GPIO, network communications, and support for various serial ports.

One method to carry out a software retrofit is to encapsulate legacy communications within an OPC UA server. One legacy communication protocol of interest is OPC Data Access (OPC DA), which is part of OPC Classic, a precursor to OPC UA. Many legacy machines support OPC DA; however, access to OPC DA data cannot be obtained directly from a UA server. The main method of accessing this data through a UA server is the

implementation of a UA wrapper, shown in Fig. 2. This method involves exposing the UA server to a DA client, which then maps the data from the connected DA server into the UA server. This allows UA clients from the wider network to access the translated DA data [9]. Unified Automation offers a subscription-based program, UaGateway, which does this data translation through the wrapper method [14]. However, this is costly and was not found to be widely used in research.

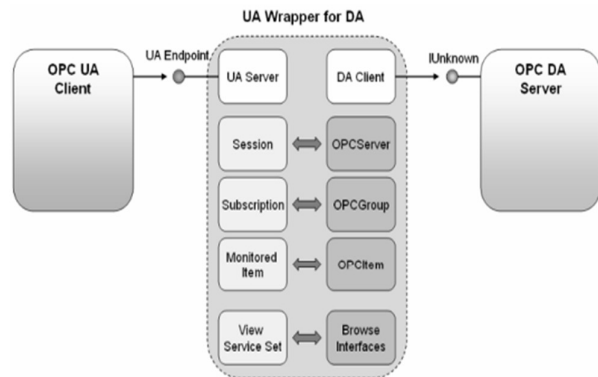


Fig. 2. OPC UA Wrapper for access to OPC DA servers [9]

## 2.3. Communication and Control of KUKA Robots

Obtaining data at the field level before it can be transferred between machines requires integration with the different proprietary communication and control protocols involved. In particular, the lab where this research has been carried out consists of two KUKA robots. KUKA robots have multiple different protocols and communication methods that can be used to read and write operational data, which can be integrated with OPC UA.

### 2.3.1. KUKAVarProxy and JOpenShowVar

One method uses an open-source program, KUKAVarProxy (KVP). KVP utilizes KUKA CrossCom to access variables within the KUKA Software System (KSS) to monitor and control the robot's operation. To connect with external devices, KVP has a server running on the Windows environment of the KUKA Robot Controller (KRC) that can be accessed over the network using a client connection [15], [16].

JOpenShowVar acts as a client to the KVP server to access and interpret KUKA Robot Language (KRL) variables [16]. Given that this method only allows modification of program and system variables, the extent of control over the robot is dictated by the program running on the KRC. This program must be written in KRL, a somewhat limited language with no mechanism for including third-party libraries, limiting its research potential.

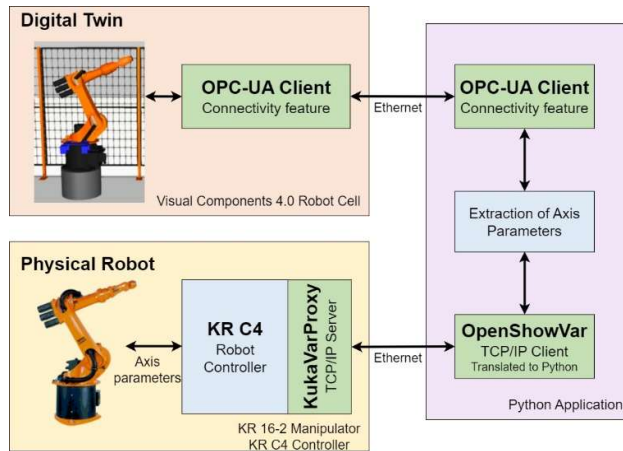


Fig. 3. KUKAVarProxy and OpenShowVar used to establish OPC UA Communication [17]

In previous research, a detailed method of KUKA robot control and monitoring that uses KVP to achieve OPC UA communications is outlined. This method implements KVP on the KRC to control and monitor the robot based on the instructions from an OPC UA client, as shown within Fig. 3. A Python application acts as the middleware, retrieving information from the KVP server using an OpenShowVar client, extracting the variable data, and publishing it to the OPC UA server [17].

This KVP server operates on the Windows environment of the robot controller, meaning it has a lower priority than the real-time control tasks, reducing its real-time capabilities. Additionally, the software is open source and has not been officially verified, utilizing backdoor methods that are actively discouraged by KUKA.

### 2.3.2. Ethernet KRL

Another method of communication is the KUKA.Ethernet KRL (KEK) XML add-on technology package [18]. KEK is a method of data transmission between external devices and the robot controller via client-server relations over an Ethernet TCP/IP network. The robot controller acts as a client, while the external device acts as a server. Communication between these two devices occurs through the transmission of XML strings. The XML structure only allows the transfer of a limited number of variable types, such as floating-point numbers, integers and boolean values. While this does not allow for full control of the robot, the robot can be programmed to handle these different inputs to trigger different subprograms like the KVP method. The KEK package includes various functions for establishing and managing connections, sending and receiving data and handling errors. Some drawbacks of this method that they point out are that the Ethernet KRL Interface (EKI) is not real-time capable, limiting its application in time-critical operations [19]. As a result, KEK XML is better suited for scenarios where reliable

communication and a simple level of data exchange are required, such as system monitoring and data logging.

### 2.3.3. Robot Sensor Interface (RSI)

RSI is a KUKA proprietary software intended for real-time adjustments to the Tool-Centre-Point position through sensor input while the robot runs a KRL program [19]. However, this can be used as a robot control method with positional signals sent through a User Datagram Protocol (UDP) connection from a PC. These signals can be processed by a PID controller for closed-loop control over the Tool-Centre-Point [19]. As this robot control method was not intended for the general operation of the robot, the RSI generally ignores most safety systems, such as maximum velocity. When a KUKA robot controller (KRC) receives a new point, the robot will attempt to reach it within the 12ms interpolation period, which can lead to sudden, unpredictable movements. For this reason, RSI must be implemented with caution. RSI is an optional, paid package that must be installed on the KRC, meaning its capabilities are only relevant if installation is feasible and cost-effective.

### 2.3.4. KUKA.OPC Server

For KUKA robots, the add-on package KUKA.OPC Server enables OPC DA communications directly with the KUKA robot controller, facilitating the reading and writing of industrial robot variables like KVP or KEK [20]. While OPC DA communications are typically limited to Windows as they require the use of Distributed Component Object Model (DCOM) technology, this package supports XML DA, enabling communication across different operating systems. Although implementations of this package will exist to control and monitor the KUKA robots, no documented examples have been identified within the literature.

It is worth noting that there is an updated version of this package with significantly improved capabilities called KUKA.DeviceConnector. It provides access to data via MQTT or OPC UA. While an opportunity for temporary licensing of this package was presented, this research is focused on implementing Industry 4.0 on legacy systems where software upgrades like this might not be compatible with existing hardware or not financially viable.

## 3. Methodology

This section outlines the general methodology behind machine retrofitting and the communication architecture. This research builds on individual technologies introduced in the related work section, such as OPC UA and the concept of digital retrofitting. It differs from previous implementations mainly in how the digital retrofit is applied to the different devices, as well as in how OPC XML-DA is used for control over the robot movements.

### 3.1. Digital Retrofitting of Legacy Systems

The literature review outlined a general 4-step process as part of introducing the AAS. To meet the needs of the machines involved, the steps are generalized into an approach consisting of three layers: the communication layer, the integration layer, and the asset layer, as shown in Fig. 4. This naming convention is based on the Reference Architectural Model Industrie 4.0 (RAMI 4.0), which was introduced in 2015 to provide a structured method of describing Industry 4.0 between participants [21]. This method is then implemented on each machine in the LISMS, described in section 4.

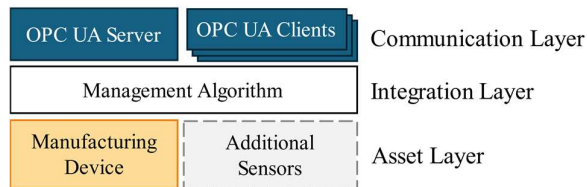


Fig. 4. Layered framework for digital retrofitting legacy machines

### 3.2. Communication Layer

The communication layer specifies how the device connects to others in the network. As part of the digital retrofit, each device will be set up with an OPC UA server. This allowed the server-to-server communication architecture to be used, where each server included embedded clients so that each machine could have two-way communication with all connected devices. This keeps the communication architecture modular and independent as each device can be operated individually, and the addition of a device simply requires a client to be set up and connected to the endpoint of the new device's OPC UA server.

Each server is structured to provide an in-depth information model of the device in accordance with OPC Foundation specifications [22], organizing static information about its manufacturer to dynamic information such as the current state of the device. Each OPC UA server also contains methods that enable external devices to invoke tasks via a simplified interface.

### 3.3. Integration Layer

The integration layer manages the data flow between the asset layer and the communication layer and then uses this information to determine control commands for that device and external devices. This includes continuously updating the status of the manufacturing device and sensor information to the device OPC UA server, executing hardware control commands based on method calls, and carrying out M2M communication. To facilitate these additional computations and provide a standardized approach across the manufacturing cell, a non-invasive control board will be installed for each device. The

Raspberry Pi is chosen as the control board due to its previous use as an Industry 4.0 retrofitting device [11], [12] established use in research, diverse connectivity options and adequate computational power.

### 3.4. Asset Layer

The asset layer comprises of both existing legacy hardware and any additional sensors required for the retrofit. These sensors aim to provide sufficient detail such that the state of that device or the materials within can be accurately determined rather than assumed. This could involve positional sensors such as encoders, photoelectric sensors, and cameras. The new control methods for the various hardware must consider safety throughout. This means utilizing existing industry control methods where possible, as well as layers of safety systems and fail-safe states.

## 4. Implementations

The retrofitting methodology outlined above is applied to the machines in the University of Auckland's (UoA) Laboratory for Industry 4.0 and Smart Manufacturing Systems (LISMS), as seen in Fig. 5. This includes a serpentine conveyor, a linear conveyor, and two KUKA robots; a KR10 and KR16. The initial system is visualized in Fig. 6 below, with communication limited to IO through a central PLC.



Fig. 5. UoA LISMS Manufacturing Cell

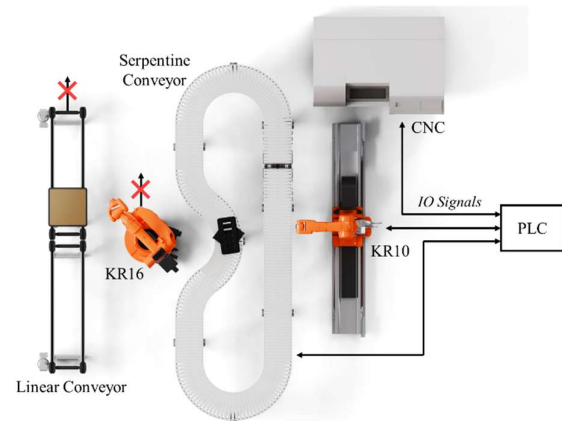


Fig. 6. Initial communications of the LISMS



#### 4.1. KUKA Robots

Both KUKA robots are capable of a range of movements while providing detailed diagnostic and status information on the KRC. The challenge is integrating these assets with the rest of the manufacturing network such that it can be controlled via a simplified interface and monitored externally to affect and respond to changes in other hardware in the manufacturing cell.

##### 4.1.1. Asset Layer

To retrofit the KUKA robots, a strategy to access the KRC's data and controls must be defined. Ultimately this determines what information will be available to the OPC UA model and the capabilities of the OPC UA methods.

##### 4.1.1.1. Asset Integration Methods

RSI provides the finest degree of control in real time of the different methods; however, due to the age of the KRC2, RSI is not installable. Ethernet KRL is also not installed on either KRC and cannot be done due to budget constraints, leaving KVP and KUKA.OPC Server as the options considered.

KVP, in conjunction with a translating middleware, has demonstrated its effectiveness for near-real-time communication with KUKA robots; however, it operates through unconventional methods to access robot variables [17]. These methods compromise the robot's compliance with industry safety standards as KVP is not an industry-certified technology.

In contrast, the KUKA.As an official KUKA product, OPC Server package is certified for industry use, ensuring that robot control can be achieved within a safety-compliant framework. Additionally, using this package on the KUKA robots to establish OPC UA communication is an area that lacks research. As the two KUKA robot controllers in the LISMS are already configured with the KUKA.OPC Server package, it can be used as a unique way to enable OPC UA communications.

##### 4.1.1.2. KUKA.OPC Server Communications

The KUKA.OPC Server offers both OPC DA and OPC XML-DA configurations. OPC XML-DA can communicate with any operating system since it utilizes XML to package data, while OPC DA uses DCOM, which is Microsoft Windows dependent. To allow cross-compatibility between operating systems, this package is implemented using OPC XML-DA.

The implementation requires an external client to read, write, and ideally subscribe to variables on the KRC server using Python. As an up-to-date package that was free [23], [24], [25] could not be found, a script capable of reading and writing to the OPC XML-DA server is then created.

To create this library, individual read and write calls are made to the KRC's OPC XML-DA server, with the corresponding IP and port being monitored in Wireshark. The, and the resulting requests and responses are then analyzed and replicated as follows.

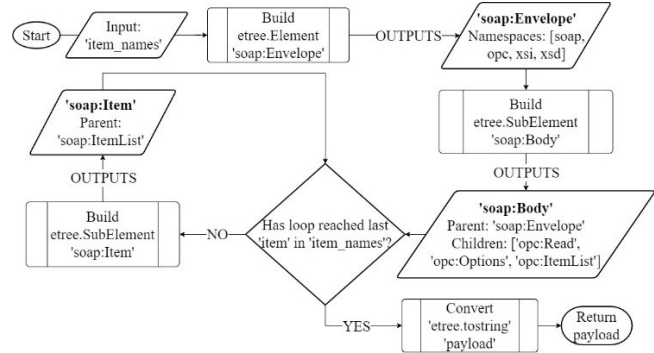


Fig. 7. Flowchart for the generation of a read request

From the structure of read and write requests, both have the same general XML format with namespaces imported for the correct XML tree elements. The 'SOAP' namespace is used for structuring the overall request, the 'OPC' namespace for generating the read/write structure, and the 'XSI' and 'XSD' namespaces for specifying the datatypes. The XML requests comprise a 'SOAP' body structure within a 'SOAP' envelope. Within the body structure, there is an 'OPC' read and write structure for the respective requests, which itself is made up of an 'Options' and 'ItemList' branch, containing information about how the response should be formatted, as well as the OPC objects to read/write to. The formulating of a read request can be seen in the flowchart in Fig. 7, with a write request formed in a similar logic pattern.

From the structure of the read-and-write responses, they have a similar XML format to the requests. Since the format is in an XML tree format, the response can be traversed by searching for child elements of each branch. For a read response, this follows the traversal pattern: Root → ReadResponse → RItemList → Items → Item → Value. Once at the 'Items' branch, a for loop is used to traverse to and store the value of each 'Item', which is then stored in an array and returned. If no items are found, the read response fails.

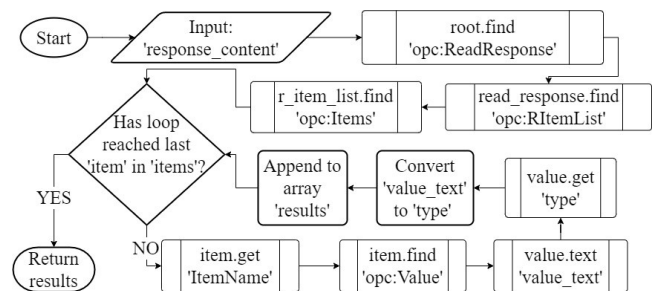


Fig. 8. Flowchart for the interpretation of a read response

For a write response, this follows the traversal pattern: Root → WriteResponse → RItemList → Items → Item → ItemName and ResultID. Once at the ‘Items’ branch, a for loop is again used to traverse to and store the name and result for the value written to that ‘Item’, which is then stored in an array and returned. If no items are found, the write response fails. The interpreting of a read response can be seen in the flowchart in Fig. 8, with a write response interpreted in a similar logic pattern.

#### 4.1.1.3. Practical Implementation

In this implementation, the OPC DA XML client is hosted by a Raspberry Pi 5, which communicates with the KRC’s OPC DA XML server as seen in Fig. 9.

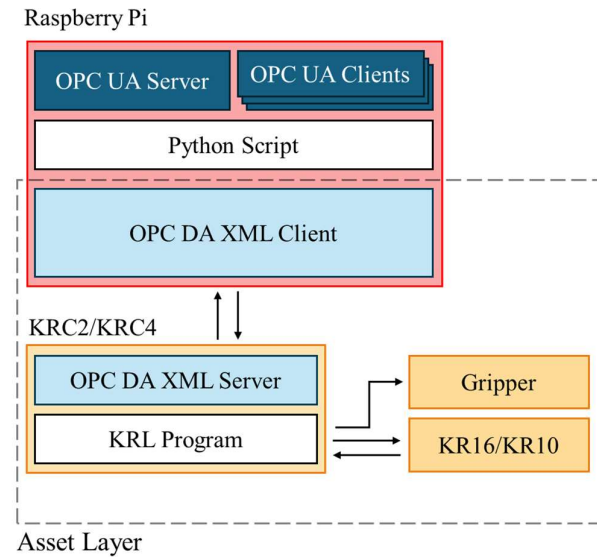


Fig. 9. Hardware specific implementation of retrofitting framework for KUKA robots

By modifying program variables, the client can trigger different subprograms on the KUKA robot to invoke specific movements and cause the robot to react to the actions of the other industry devices. At the same time, system variables such as the robot's current position can be constantly read, providing near real-time monitoring.

#### 4.1.2. Communication Layer

As previously established, communications are facilitated by server-to-server OPC UA communications using embedded OPC UA clients.

To improve interoperability and industry compliance, this OPC UA server is structured in accordance with the OPC Foundations specifications for robotic devices, as seen in Fig. 10.

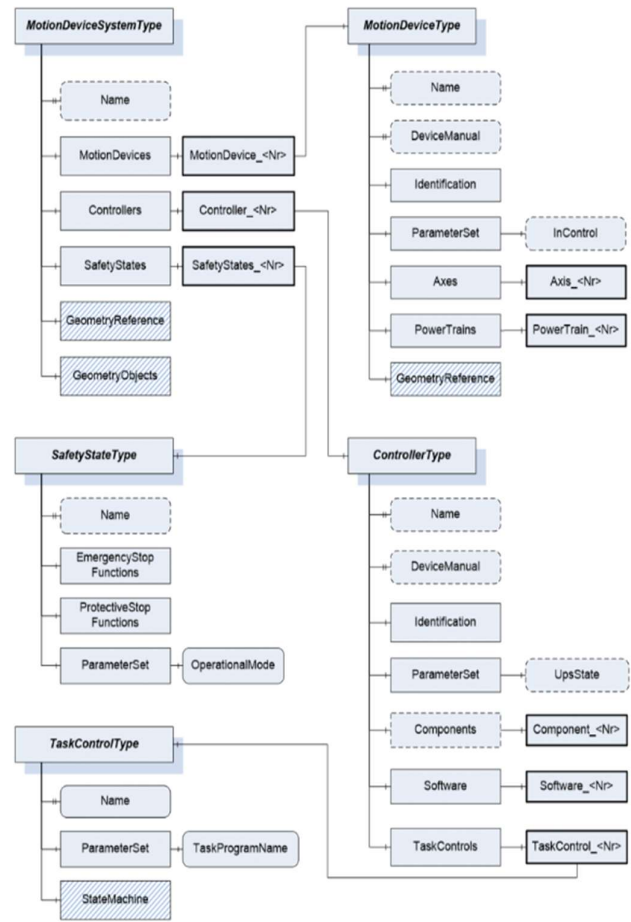


Fig. 10. OPC Foundations Information Model for Robotics Companion Specification [26]

#### 4.1.3. Integration Layer

With communication to the asset layer established and the OPC UA information model configured, this section now examines the implementation of these capabilities to develop a program to demonstrate and test their capabilities. This manifests as the development of an OPC UA method called *MovePart*, which moves a part between two points. This method is chosen since it includes multiple stages of operation dependent on surrounding devices and movements to a dynamic range of positions. These functionalities require M2M communication and flexible robot operation, demonstrating key Industry 4.0 skills.

The *MovePart* method is designed to pick up a part from one input position and drop off the part at the other input position. For example, on the KR10, the pickup and drop-off inputs might be “serp\_conv” (serpentine conveyor) and “cnc”. The exact part position and each stage of the robot’s operation is determined based on information gathered from M2M communications via OPC UA.

Each of these movements comprises a pre-defined component and a dynamic component, as seen in Fig. 11. The pre-defined motions consist of large movements between separate zones, such as the linear conveyor and the

serpentine conveyor. These large movements are pre-defined since the position of the Tool Centre Point (TCP) can be controlled, but not the rest of the robot arm, without implementing path planning algorithms. The undefined and unpredictable movement of the robot arm poses a significant safety hazard. To address this, each movement path must be individually tested and eventually predefined within the program. Although this limits the areas the robot can reach, additional movements can be quickly added to expand this by creating a new function that mimics the existing control architecture.

The dynamic component of the movement enables the robot to pick up the part from an approximately continuous range of positions. These coordinate-based movements are executed using a linear movement within a pre-specified zone so that the robot will move predictably while ensuring it cannot hit anything.

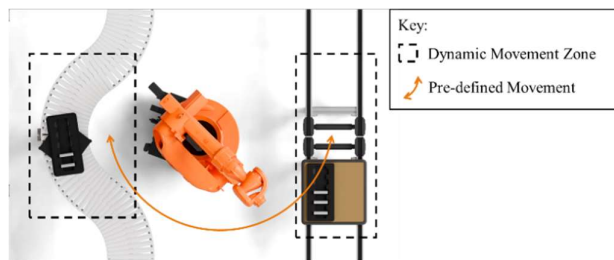


Fig. 11. Dynamic and pre-defined movements of KR16 between serpentine conveyor and linear conveyor

A simple sub-program was created for each KUKA robot to achieve movements to any coordinate. The KRC is programmed so that any movement function cannot take in a variable as an input and will only accept constants. Therefore, to allow variable-driven movements, movements are then hard-coded using a conditional logic structure. For example, given a target coordinate of  $X = 98.2$ , it will satisfy the condition  $97.5 < X \leq 98.5$ , so the robot will execute a command to adjust its position using a linear movement to  $X = 98$ .

#### 4.2. Conveyors

Both conveyors (serpentine and linear) underwent similar levels of digital retrofit, following the framework outlined in section 3.

##### 4.2.1. Asset Layer

The original hardware for both conveyors provided no method of monitoring the actions of parts on the conveyor. For this reason, discrete, real-time part tracking is done through photoelectric sensors mounted at loading points on each conveyor. A computer vision system is used to monitor the tray and part precisely. Additionally, control commands must be passed to the manufacturing hardware from the Raspberry Pi 4 that determines a response based on sensor

data, machine state, method calls and surrounding hardware.

For the linear conveyor, both the photoelectric sensors and the linear conveyor's controller are interfaced to a Raspberry Pi 4 controller through buffered GPIO inputs on a Pimoroni Automation hat, as shown in Fig. 12. The IP camera was interfaced with the Raspberry Pi 4 through an ethernet connection, with calibration carried out to remove radial and tangential distortion of the image feed.

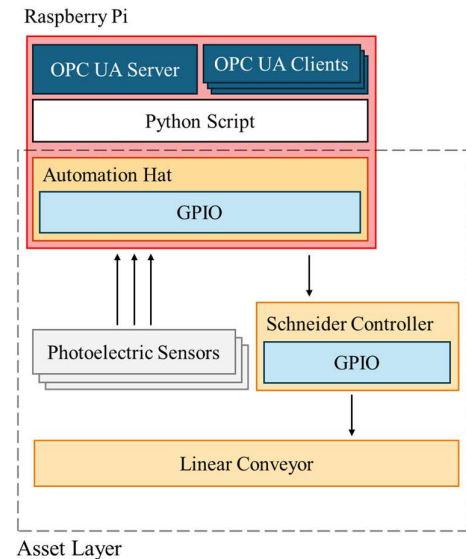


Fig. 12. Hardware specific implementation of digital retrofit for the linear conveyor (excluding computer vision)

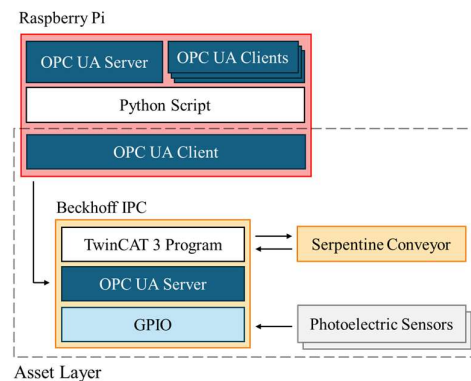


Fig. 13. Hardware-specific implementation of digital retrofit for the serpentine conveyor (excluding computer vision)

The serpentine conveyor had already been implemented into the manufacturing cell utilizing a sophisticated Beckhoff CX2033 IPC, extended by an EtherCAT EK1100 module. This module provided IO ports to which the photoelectric sensors were connected. Using the TwinCAT 3 program's OPC UA communications, a Raspberry Pi 5 was able to access control of the serpentine conveyor's movement as well as monitor the status of the sensors. In its original state, the Beckhoff IPC was essentially an unprogrammed control board so a digital retrofit would



need to have been carried out on this device. Rather than implementing the retrofitting strategy directly on the IPC, a Raspberry Pi 5 is added for an additional translation layer with the machine, such that the implementation is standardized across the manufacturing hardware. Fig. 13 shows this hardware specific implementation for the serpentine conveyor. A USB camera is connected to an additional Raspberry Pi 5, where an image processing and part tracking algorithm was run.

#### 4.2.2. Integration Layer

The integration layer acting on the Raspberry Pi 5 takes in outputs from the sensors within the asset layer as well as data from surrounding devices through the communication layer to control the movement of the conveyors.

For the linear conveyor, this involved GPIO based control over the motor's direction, with additional sensors providing the necessary positional data to keep the tray on the conveyor. Methods on the OPC UA server invoked control commands to move the tray to a desired location. This control is done through utilizing multithreading, shown in Fig. 14.

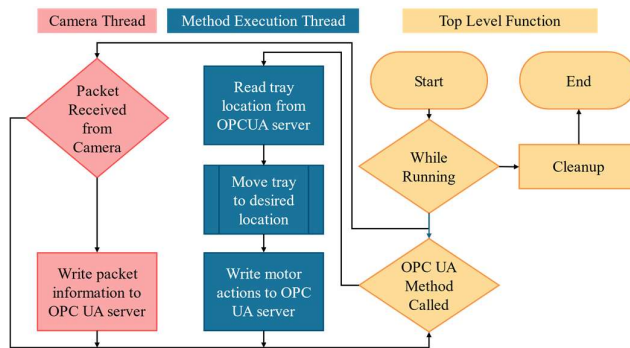


Fig. 14. Flowchart of main control architecture for the linear conveyor

The main loop is responsible for keeping the server alive, and when run, it instantiates threads for incoming data from the camera and client connections. When a client calls a method, an additional thread is made that runs that method until completion.

The serpentine conveyor has a slightly different integration layer. Since it has an IPC, this does the field level control over moving the conveyor. The Raspberry Pi 5 is added for interfacing the camera and ease of integration with the rest of the network, elaborated on in section 4.2.3.

#### 4.2.3. Communication Layer

Following the server-to-server pattern, the OPC UA server for both conveyors are constructed with embedded clients for each external device connected. This allows two-way communication, with clients being used to invoke methods on other devices, and external clients invoking methods on the conveyors OPC UA server. The servers are

set up such that all communication that alters the state of the conveyor is done through methods, with all variables in a read-only state. Changing these variables is done internally, so that if the 'RequestLoad' method is called on the linear conveyor, the tray will be moved to the load state. This maintains safety standards, so that any external connection made cannot alter operational variables directly.

Data organization is another part of the communication layer, done through an information model. For both conveyors the *MotionDeviceSystemType* was implemented as part of the Robotics specifications. While a linear conveyor may not meet some definitions of a robot, the *MotionDeviceSystemType* is the closest matching specification. This type includes information like the motion device, its axial movements, the power train, the controller and tasks, encapsulating much of the information required of this device [26]. This type is then extended for the additional hardware added to the conveyor, resulting in the following types added:

- *PESensorType* – Represents the photoelectric sensors, detailing their current state, device name, serial number, and model.
- *ConveyorMotorType* – Represents the conveyor's motor, including information such as the manufacturer, model, serial number, and motor direction.
- *ComputerVisionSystemType* – Represents the camera system, including information such as its IP address and manufacturer, along with positional data about the tray and part.

The information model that the implemented server extends is shown in Fig. 9 in section 4.1.2.

## 5. Results and Discussion

The work done in the UoA's LISMS has culminated in a range of devices with improved Industry 4.0 capabilities facilitated by OPC UA communications and distributed intelligence as seen in Fig. 15. The two KUKA robotic arms, the serpentine conveyor and linear conveyor, have each been configured with an OPC UA server providing an interface for improved device monitoring and control. While the CNC machine should have been included in this, time limitations mean its inclusion will be part of a future project. With each piece of manufacturing hardware represented by a separate OPC UA server (aside from the two KUKA robots), the system provides a modular framework capable of expanding the capabilities of each manufacturing device. Utilizing OPC UA, each device is capable of machine-to-machine communication enabling devices to monitor each other's progress throughout different stages to create more dynamic operation of the cell. Additionally, the modular setup allows each device to call each other's methods distributing control across the manufacturing cell.

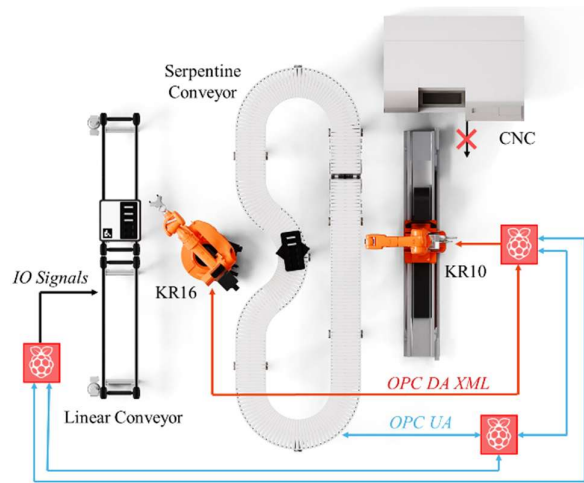


Fig. 15. Final communication architecture in the LISMS

Together these devices can successfully pass a part across the various stages of the manufacturing cell providing detailed information across each of these stages. To highlight the capabilities of the manufacturing cell, the system has been programmed to loop, passing the part back and forth across the cell without any form of central oversight as seen within Fig. 16.

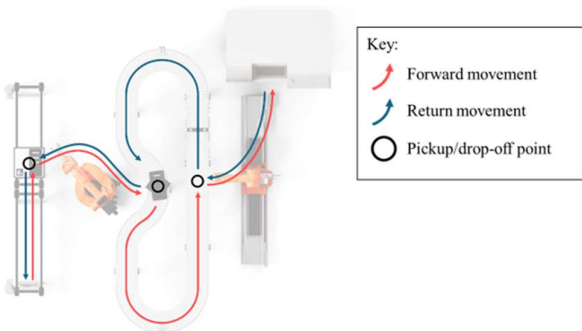


Fig. 16. Movement of part across the manufacturing cell

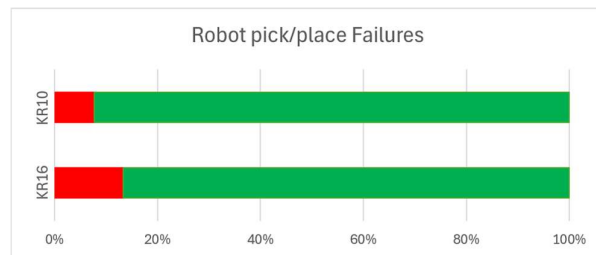


Fig. 17. Graph showing robot pick/place failure rates

In the duration of the recorded loop, shown in the video link [27], approximately 7 complete cycles were run. Throughout these cycles, the percentage of movements that had a pick or place failure by the KR10 was 8.3%, and for the KR16 the failure rate was 15.4%, visualized in Fig. 17. These failures are attributable to the object tracking systems, as the point of failure is on the coordinate passed to each robot. In terms of delays between movements, these

accumulated to 13.9% of the total time the loop was running. For around 50% of these, they were directly attributable to the breakdown in the OPC XML-DA connection with the KUKA robots. The other 50% was caused by bugs in the code involving how methods were invoked by clients. Fig. 18 shows an example of a failure where the part was placed on an angle in the tray. This failure then gets carried through, as the robot will continue to pick up the object on that angle. A potential solution to this problem would be to use a higher resolution camera. This would allow the object tracking algorithm to be developed further, as the accuracy of it is mainly limited by the camera quality. Furthermore, in the case where a part was placed incorrectly, the algorithm could be changed to handle these edge cases. An incorrectly placed part would have a slightly different area which could be used to rotate and move the robot's end effector accordingly. Finally, a larger tolerance for error could be introduced on the tray through adding chamfered edges to funnel the part into the correct position. This would allow part placement to be less dependent on the accuracy of the coordinates being obtained by the object tracking algorithm.



Fig. 18. Example place failure on the linear conveyor tray

The breakdown of the OPC XML-DA connection with the KUKA robots refers to a continuous unsuccessful read or write of/to a variable on the robots DA server. This delay came from the read/write command being set in a while loop, which exited on the condition the client successfully completed that command. Due to this, delays would accumulate if the connection was down.

Otherwise, methods invoked by clients would be transferred to a new thread, whose future would be returned to the running thread on the server whose method got invoked. From the logs, a common error often seen is that the future would return its value without the method being carried out. Due to this, the method would be called again by the client to retry invoking the movement, causing the accumulation of more delays.

The communication delay problem is harder to solve as the problem requires pinpointing. It is assumed that the problem is either in how the methods are called on each server or in the translation to how the variables get written

or read on the robots DA server. Once the exact point of failure is known, a solution would be able to be developed.

## 6. Conclusions

This work proposes a general digital retrofitting approach that can be applied to a wide range of legacy devices. The proposed framework is structured around the RAMI4.0 model and expands on the 4-step approach discussed in [10]. Following the Industry 4.0 standard, OPC UA is used as the standardized communication protocol, with all devices involved extended by an OPC UA server, organized through the OPC Foundations information modelling standards. A focus was put in developing and implementing a modular communication architecture, which was established through distributed control. This allows each machine to operate independently, with communication pathways kept to a minimum such that they can be interchangeable.

The implemented communication architecture was then tested by running the manufacturing cell in a loop, simulating a production environment. While the resulting system has some communication delays and pick/place faults, it is functional and successfully demonstrates distributed control through an OPC UA based communication architecture. This project lays the groundwork for future work in the LISMS, with plans in place for the continuation of the work demonstrated in this paper. The communication delay and pick/place problems outlined in the discussion will be addressed, and additional functionality will be added with a CNC machine tool and a more functional user interface for the system.

## Acknowledgements

We extend our heartfelt thanks to Trishit Ghatak, our lab technician, whose technical expertise and willingness to go above and beyond consistently supported our work. His knowledge of KUKA robotic systems and lab machinery, along with his daily enthusiasm, made this project both successful and enjoyable.

We would also like to thank Tang Ji for generously sharing his knowledge of the lab and manufacturing cell, which saved us considerable time and helped us overcome technical challenges efficiently.

## References

- [1] Zhong, R.Y., Xu, X., Klotz, E., Newman, S.T. "Intelligent Manufacturing in the Context of Industry 4.0: A Review". (2017) Engineering, 3 (5), pp. 616–630. DOI: 10.1016/J.ENG.2017.05.015
- [2] L. Y. Yi et al., "Readiness for and adoption of Industry 4.0 among small and medium sized enterprises in the Malaysian furniture industry," *Bioresources*, vol. 16, no. 4, pp. 8289–8308, Oct. 2021, doi: 10.15376/biores.16.4.8289-8308.
- [3] A. M. Deshpande, G. G. Keskar, S. A. Pradeep, S. Farahani, and S. Pilla, "Composites 4.0: Enabling the Modernization of Legacy Manufacturing Assets in South Carolina," in *Volume 1: Additive Manufacturing; Advanced Materials Manufacturing; Biomaterials; Life Cycle Engineering*, American Society of Mechanical Engineers, Jun. 2023. doi: 10.1115/MSEC2023-104904.
- [4] D. Tantscher and B. Mayer, "Digital Retrofitting of legacy machines: A holistic procedure model for industrial companies," *CIRP J Manuf Sci Technol*, vol. 36, pp. 35–44, Jan. 2022, doi: 10.1016/j.cirpj.2021.10.011.
- [5] A. Alqoud, D. Schaefer, and J. Milisavljevic-Syed, "Industry 4.0: a systematic review of legacy manufacturing system digital retrofitting," *Manuf Rev (Les Ulis)*, vol. 9, p. 32, Nov. 2022, doi: 10.1051/mfreview/2022031.
- [6] S. Cavalieri, "A Proposal to Improve Interoperability in the Industry 4.0 Based on the Open Platform Communications Unified Architecture Standard," *Computers*, vol. 10, no. 6, p. 70, May 2021, doi: 10.3390/computers10060070.
- [7] "VDMA – Machinery and Equipment Manufacturers Association – Medical Technology Forum / German Pavilion, MD & M West 2024." [Online]. Available: <https://mdmwest.german-pavilion.com/en/sites/contact/2356>
- [8] VDMA, "OPC UA." [Online]. Available: <https://www.vdma.org/opc-ua>
- [9] W. Mahnke, S.-H. Leitner, and M. Damm, *OPC Unified Architecture*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009. doi: 10.1007/978-3-540-68899-0.
- [10] J. I. García, R. E. Cano, and J. D. Contreras, "Digital retrofit: A first step toward the adoption of Industry 4.0 to the manufacturing systems of small and medium-sized enterprises," *Proc Inst Mech Eng B J Eng Manuf*, vol. 234, no. 8, pp. 1156–1169, Jun. 2020, doi: 10.1177/0954405420904852.
- [11] S. Lass and N. Gronau, "A factory operating system for extending existing factories to Industry 4.0," *Comput Ind*, vol. 115, p. 103128, Feb. 2020, doi: 10.1016/j.compind.2019.103128.
- [12] R. H. S. J. Fusic, J. Cornelius K S R, S. M., and A. M., "Case study on Integrating Legacy Devices in Industry 4.0 framework using OPC UA," in *2023 International Conference on Energy, Materials and Communication Engineering (ICEMCE)*, IEEE, Dec. 2023, pp. 1–6. doi: 10.1109/ICEMCE57940.2023.10434133.
- [13] T. Lins and R. A. R. Oliveira, "Cyber-physical production systems retrofitting in context of industry 4.0," *Comput Ind Eng*, vol. 139, p. 106193, Jan. 2020, doi: 10.1016/j.cie.2019.106193.
- [14] M. A. O. Pessoa, M. A. Pisching, L. Yao, F. Junqueira, P. E. Miyagi, and B. Benatallah, "Industry 4.0, How to Integrate Legacy Devices: A Cloud IoT Approach," in *IECON 2018 - 44th Annual Conference of the IEEE Industrial Electronics Society*, IEEE, Oct. 2018, pp. 2902–2907. doi: 10.1109/IECON.2018.8592774.
- [15] I. Eriksen and M. H. Arbo, "Setup and Interfacing of a KUKA Robotics Lab," 2017.
- [16] F. Sanfilippo, L. I. Hatledal, H. Zhang, M. Fago, and K. Y. Pettersen, "JOpenShowVar: An open-source cross-platform communication interface to Kuka robots," in *2014 IEEE International Conference on Information and Automation (ICIA)*, IEEE, Jul. 2014, pp. 1154–1159. doi: 10.1109/ICInfA.2014.6932823.
- [17] A. Øvern, "Industry 4.0-Digital twins and OPC UA," NTNU, 2018.
- [18] "KUKA Xpert." [Online]. Available: [https://xpert.kuka.com/service-express/portal/project1\\_p/document/kuka-project1\\_p-common\\_PB3582\\_en?context=%7B%22filter%22%3A%7B%7D,%22text%22%3A%22KUKA%20Ethernet%20KRL%22,%22page%22%3A1,%22useExpertQuery%22%3A0%7D](https://xpert.kuka.com/service-express/portal/project1_p/document/kuka-project1_p-common_PB3582_en?context=%7B%22filter%22%3A%7B%7D,%22text%22%3A%22KUKA%20Ethernet%20KRL%22,%22page%22%3A1,%22useExpertQuery%22%3A0%7D)
- [19] M. Mehner, N. Matkovic, E. Mühlbeier, D. Mayer, and J. Fleischer, *Evaluation of external control of KUKA Industrial Robots for laboratory and prototype environments*. VDE Verlag, 2023.
- [20] *KUKA OPC Server 3.1*. KUKA System Technology, 2010.

- [21] M. Hankel and B. Rexroth, “Industrie 4.0: The Reference Architectural Model Industrie 4.0 (RAMI 4.0),” Nov. 2015.
- [22] O. P. C. Foundation, “Nodesets.” [Online]. Available: <https://reference.opcfoundation.org/nodesets>
- [23] “python-opc.” [Online]. Available: <https://pypi.org/project/python-opc/>
- [24] “PyOPC,” 2006. [Online]. Available: <https://sourceforge.net/projects/pyopc/files/>
- [25] “Create OPC clients for OPC-DA (Data Access) - OPC Labs.” [Online]. Available: <https://www.opclabs.com/products/quickopc/opc-specifications/opc-xml-da>
- [26] “Robotics - Vertical integration.” [Online]. Available: <https://reference.opcfoundation.org/Robotics/v100/docs/>
- [27] L. Katavich, “Project Demonstration | Retrofitting Legacy Systems for Industry 4.0,” Nov. 2024. [Online]. Available: <https://www.youtube.com/watch?v=7sO2M8nO144>